Algorithm 1 Gantry: A* Search (Version 0)

Input: grid width and height; blocked cells; homed (yes/no); Start = $(0, 0)$; End
Output: Path from Start to End, or an error message and an empty Path
0. Check inputs
if not homed then
    print "Error: not homed."; return an empty Path ▷ **moved first: without homing, (0, 0) means nothing**
end if
if End is outside the grid **then**
    print "Error: target out of bounds."; **return** an empty Path
end if
if End is blocked **then**
    print "Error: target blocked."; **return** an empty Path
end if
1. Setup
Queue ← empty; counter ← 0
Parents ← empty; Distance ← empty ▷ dictionaries (hash maps)
Visited ← empty ▷ HashSet
every vertex not in Distance has distance ∞
Distance[Start] ← 0; Parents[Start] ← null
enqueue $(f(\text{Start}), h(\text{Start}), \text{counter}, \text{Start})$; counter ← counter +1 ▷ ties: lower h, then lower counter
2. Loop
while Queue.count > 0 **and** End is not in Visited **do**
    pop the lowest entry from Queue; curr ← its vertex
    if curr is in Visited **then**
        skip to the next loop iteration ▷ old duplicate
    end if
    for each edge from curr to n (right, up, left, down) **do**
        if n is in Visited **then**
            skip to the next edge ▷ safe: h never overestimates
        end if
        new-dist ← Distance[curr] + edge-dist
        if new-dist < Distance[n] **then**
            Distance[n] ← new-dist; Parents[n] ← curr
            enqueue $(\text{new-dist} + h(n), h(n), \text{counter}, n)$; counter ← counter +1
        end if
    end for
    add curr to Visited
end while
if End is not in Visited **then**
    print "Error: no route found."; **return** an empty Path
end if
3. Build Path
Stack ← empty; v ← End
while v is not null **do**
    push v onto Stack; v ← Parents[v]
end while
Path ← empty
while Stack is not empty **do**
    pop from Stack and add the vertex to the end of Path
end while
return Path

Red: added or changed from your submission.)